EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Illustrative gym-chain scenario compared with the Gym Membership Management System

Submitted by	PRANAV SRINEVAS V	**Register No.**	241401068
Application	Gym_Membership_Management_System (Zoho Creator)	**Department**	B.Tech CSBS

Note: the enterprise scenario in this report is an illustrative composite based on commonly documented low-code adoption challenges. It is not a record of a specific named company, and the figures given are illustrative.

1. Abstract

Low-code platforms such as OutSystems, Mendix, Microsoft Power Apps, Appian and Zoho Creator promise faster delivery through visual modelling and prebuilt components. They work well for departmental tools, workflow automation and prototypes, such as the Gym Membership Management System. When applied to complex, large-scale enterprise systems their limitations become significant. This case study follows an illustrative scenario in which a national gym chain tries to rebuild its whole membership, billing and access platform on a low-code platform, identifies the limitations that emerge, and compares them with the limitations met in the author's own gym project.

2. Introduction

Enterprises must deliver digital solutions quickly while IT talent is scarce, and low-code promises shorter time-to-market, lower initial cost and the involvement of business users. Speed in the early stages does not always lead to long-term success: complex use cases involve high transaction volumes, intricate rules, legacy integration, regulation and long lifecycles. This report assesses where low-code falls short under such conditions.

3. Background: What is low-code?

A low-code platform provides a visual environment where applications are assembled with drag-and-drop interfaces, model-driven logic, connectors and templates. Custom code can be added for exceptions, but the runtime, data layer and deployment pipeline are managed by the vendor. These strengths are also the source of its limits at scale.

4. Case scenario: FitChain national gym platform (illustrative)

FitChain (a fictional name) runs gyms in many cities with hundreds of thousands of members. It decides to replace its ageing membership system with a low-code platform covering registration, plans, class booking, payments, door access and mobile app. The first modules, member registration, plans and class schedules, are delivered quickly and resemble the gym project of this record. As the project moves to high-volume billing, thousands of promotional pricing rules, integration with turnstiles and payment gateways and strict data-protection requirements, progress slows, and the team adds custom code and

external services to work around the platform. The programme is finally restructured as a hybrid: low-code for forms, workflows and dashboards, conventional services for the core.

Complexity factors

- Volume: very high booking and payment traffic at peak hours and during promotions.
- Business rules: many plan, discount, freeze and renewal rules.
- Integration: payment gateways, access-control hardware, accounting and marketing systems.
- Compliance: data protection for personal and payment data, audit trails.
- Lifespan: a system life of ten years or more with continuous change.

5. Limitations encountered

Limitation	What happens in the scenario	Severity
Scalability and performance	Generated runtime cannot sustain peak load; little control over queries, indexes and caching; scaling depends on vendor licence tiers.	High
Complex business logic	Visual logic becomes hard to read, review, test and version when rules are deeply nested.	High
Legacy and third-party integration	Turnstiles and the billing engine need custom adapters that are fragile and tied to the platform version.	High
Vendor lock-in	Models and runtime are vendor-specific; leaving means a rewrite; renewal pricing power is lost.	High
Customisation	Needs outside the component library require workarounds and create technical debt.	Medium
Security, compliance, data residency	Audit evidence depends on vendor documentation; misconfigured access by inexperienced builders is a risk.	Medium
Governance and shadow IT	Clubs build overlapping apps; duplicate data models and inconsistent security appear.	Medium
Cost at scale	Licences grow with users, apps and environments; the pilot costs do not predict the real cost.	High
Skills and DevOps	Platform-specific talent is scarce; CI/CD, testing and source control are more constrained.	Medium

6. The same limits in the gym project

The gym application is small, which is why low-code suits it, but the same limits are visible at a small scale:

Limit	Seen in the gym application
Time-limited trial	The application header shows the trial expiring in a few days, so the project must be finished and demonstrated within that period.
Limited customisation	Dashboard charts follow the platform’s styling; a custom design would need more than the builder offers.
Logic kept in script	The workflow derives the duration from the plan name instead of reading the Duration (Months) field, so a renamed or new plan needs the script changed.
Vendor dependence	The application runs on Zoho’s cloud and its forms and scripts cannot be moved to another platform without rebuilding.
Environments	The application runs only in Development; Stage and Production would be needed for a real gym.

7. Comparison: low-code vs traditional development

Aspect	Low-code platform	Traditional development
Initial speed	Very fast for standard apps	Slower start
Complex logic	Difficult to manage visually	Flexible, testable code
Scalability control	Limited by vendor runtime	Full architectural control
Integration depth	Strong for common connectors only	Unlimited with custom code
Portability	Low (lock-in)	High (open standards)
Cost profile	Low upfront, rising licences	Higher upfront, predictable
Best fit	Workflows, forms, dashboards	Core systems, high performance

8. Lessons and recommendations

- **Assess fit before committing:** prove the hardest requirement first with a proof of concept.
- **Adopt a hybrid architecture:** low-code for forms, workflows and dashboards; conventional services for rule-heavy, high-volume cores exposed through APIs.
- **Design for exit:** document data models and logic outside the platform and keep data exportable.
- **Model total cost of ownership** over five to ten years, including licences, environments and workarounds.
- **Establish governance early:** naming and security standards, approvals and clear ownership.
- **Keep engineering practices:** version control, testing and code review as far as the platform allows.

9. When low-code is still the right choice

Low-code remains highly effective for internal workflows, data-collection forms, dashboards, self-service portals, process automation, prototypes and applications of modest scale, such as the single-gym Gym Membership Management System. The mistake in the case was not choosing low-code but expecting it to carry the most demanding core workload.

10. Conclusion

Low-code platforms speed up development and widen participation, but their abstraction has a price when scale, logic complexity, integration and compliance grow. The illustrative FitChain case shows that the best path is a deliberate hybrid: use low-code where it excels, keep conventional engineering where control matters, and plan governance and exit from the start.

11. References

- Gartner, Inc. Research publications on enterprise low-code application platforms (Magic Quadrant series).
- Forrester Research. Reports on low-code development platforms.
- Bock, A. C., and Frank, U. (2021). Low-Code Platform. Business & Information Systems Engineering, 63(6), 733–740.
- Sahay, A., Indamutsa, A., Di Ruscio, D., and Pierantonio, A. (2020). Supporting the understanding and comparison of low-code development platforms. 46th Euromicro

Conference on Software Engineering and Advanced Applications.

- Vendor documentation: Zoho Creator, OutSystems, Mendix, Microsoft Power Apps and Appian.